

Sistema Agropecuário Distribuído

Otávio Miranda Côrtes
32211BSI015

Agosto 18, 2025

Código fonte disponível no GitHub: <https://github.com/otaviomcortes/Sistemas-Distribuidos/tree/main>.

Visão geral da implementação

A Etapa 3 do projeto teve como foco a evolução da arquitetura do sistema distribuído iniciado na etapa anterior. Enquanto a Etapa 2 contemplava apenas a Central de Controle e sensores ambientais, com comunicação direta e estática, a nova etapa trouxe avanços significativos. Dentre as principais melhorias e adições realizadas nesta etapa, destacam-se:

1. Melhorias na classe SensorAmbiental: A classe que se chamava "SensorAmbiental" agora se chama "SensoresAmbientais", pois agora representa sensores ambientais diferentes que simulam o envio de informações com um intervalo pré definido, porém agora enviam mensagens estruturadas.
2. Inclusão dos Sensores de Gado: Agora foi feita a implementação da classe "SensoresDoGado" que representa sensores conectados individualmente a cada animal simulando a temperatura corporal, localização e estado do cocho. Esses sensores disparam eventos para a Central de Controle, permitindo simular situações como febre ou cocho vazio.
3. Criação do Gerenciador de Alimentadores: Este componente simula os alimentadores automáticos da fazenda. Ele recebe comandos da Central de Controle e os distribui para o alimentador correspondente ao animal, de acordo com o seu ID.
4. Atualização da Central de Controle: A Central de Controle agora processa os eventos recebidos, avalia condições (como temperatura elevada ou cocho vazio) e emite comandos para os alimentadores corretos, consultando previamente o Servidor de Nomes para encontrar o Gerenciador de Alimentadores. Na etapa 2 ela apenas recebia mensagens simples dos sensores ambientais como temperatura e umidade.

5. Padronização de Mensagens: Diferentemente da etapa 2 nesta nova etapa toda a comunicação entre os componentes segue um padrão em JSON contendo os campos tipo, origem, destino, timestamp e conteúdo. Isso torna o sistema mais organizado e extensível.
6. Introdução de um Mecanismo de Nomeação: Foi implementado o Servidor de Nomes, um processo centralizado responsável por armazenar e fornecer dinamicamente os endereços (host:porta) dos componentes do sistema. Isso elimina a dependência de configurações fixas e permite maior flexibilidade e dinamismo na comunicação.

Em resumo, a Etapa 3 consolidou a arquitetura distribuída iniciada anteriormente, introduzindo novos agentes, orquestração por nomeação e ações reativas baseadas em eventos, simulando de forma realista o funcionamento de uma fazenda inteligente conectada por meio de uma infraestrutura de comunicação robusta e escalável.

Projeto de Arquitetura de Mensagem e Fluxo de Comunicação

A arquitetura de comunicação do sistema foi estruturada com base no paradigma orientado a mensagens, permitindo a troca de informações entre processos distribuídos de forma desacoplada e flexível. Foram definidos dois tipos principais de mensagens:

- **Eventos:** Mensagens originadas por sensores (ambientais ou de gado) contendo dados do ambiente ou do estado dos animais. Essas mensagens são enviadas de forma periódica e representam a percepção do mundo real pelos sensores.
- **Comandos:** Mensagens emitidas pela Central de Controle com a finalidade de instruir componentes do sistema a realizar alguma ação específica, como alimentar um animal. São reativas aos eventos recebidos.

Estrutura das Mensagens

Todas as mensagens do sistema seguem um formato padronizado em JSON, o que garante legibilidade, interoperabilidade e facilidade de expansão futura. A estrutura básica contém os seguintes campos:

- **tipo:** Define o tipo da mensagem (ex: "evento" ou "comando").
- **origem:** Identificador do componente que enviou a mensagem (ex: "gado330", "ambiente4", "central").
- **destino:** Identificador do componente de destino (ex: "central", "alimentador4").
- **timestamp:** Data e hora da geração da mensagem, no formato ISO-8601.
- **conteudo:** Um objeto JSON contendo os dados relevantes à mensagem.

Exemplos de Mensagens

Evento enviado por um Sensor de Gado:

```
{
  "tipo": "evento",
  "origem": "gado330",
  "destino": "central",
  "timestamp": "2025-08-18T14:32:05",
  "conteudo": {
    "animal_id": "gado330",
    "temperatura_corporal": "40.2",
    "localizacao": "(55,80)",
    "cocho": "vazio"
  }
}
```

Comando emitido pela Central de Controle:

```
{
  "tipo": "comando",
  "origem": "central",
  "destino": "alimentador4",
  "timestamp": "2025-08-18T14:32:10",
  "conteudo": {
    "acao": "alimentar",
    "animal_id": "gado330"
  }
}
```

Fluxos de Comunicação no Sistema

A arquitetura proposta na Etapa 3 baseia-se em fluxos de comunicação orientados a eventos, onde os sensores enviam dados de forma contínua e a Central de Controle toma decisões com base nesses eventos. A seguir, descrevem-se os principais fluxos de comunicação, seus participantes e seus impactos.

1. Sensores Ambientais → Central de Controle

- Os sensores ambientais enviam periodicamente mensagens do tipo **evento** para a Central de Controle.
- Esses dados são usados para monitoramento ambiental geral da fazenda, podendo futuramente desencadear ações automáticas baseadas em condições climáticas adversas.
- Frequência: a cada 10–15 segundos por sensor.
- Impacto: representa a base para um fluxo contínuo de dados ambientais.

2. Sensores do Gado → Central de Controle

- Cada sensor simula informações vitais de um animal (temperatura corporal, localização, estado do cocho) e envia um **evento** único por execução.
- A Central de Controle analisa a mensagem, detecta anomalias (como febre ou cocho vazio) e decide se deve acionar um alimentador.
- Impacto: desencadeia ações automáticas e reativas do sistema, como alertas ou comandos de alimentação.

3. Central de Controle → Gerenciador de Alimentadores

- Quando detectado um cocho vazio, a Central envia uma mensagem do tipo **comando** para o alimentador correspondente via Gerenciador de Alimentadores.
- A Central consulta o endereço do Gerenciador por meio do Servidor de Nomes, garantindo flexibilidade de localização.
- Impacto: representa uma ação concreta do sistema, automatizando a alimentação dos animais.

4. Central de Controle → Usuário (via Console)

- A Central imprime mensagens de alerta quando detecta temperatura corporal acima de 39,5 °C ou ao concluir o envio de comandos para alimentação.
- Esse mecanismo pode ser futuramente substituído por uma interface de notificação em tempo real ou painel visual.
- Impacto: permite acompanhamento em tempo real e diagnóstico da operação da fazenda.

Fluxo Contínuo de Dados

O sistema atual já simula um fluxo contínuo de dados por meio dos sensores ambientais, que operam em paralelo e transmitem informações periodicamente. Essa abordagem foi projetada para suportar cenários em tempo real, o que será essencial em etapas futuras para:

- Aplicação de análises preditivas;
- Geração de relatórios em tempo real;
- Adoção de mecanismos de alarme e resposta automática;
- Detecção de padrões comportamentais e ambientais.

Esse fluxo contínuo é viabilizado pela arquitetura multithread do sistema e pelo uso de mensagens JSON, que garantem escalabilidade e desacoplamento entre os agentes.

Mecanismo de Nomenclatura do Processo

Nesta etapa foi implementado um mecanismo de nomeação centralizado, por meio do componente denominado Servidor de Nomes, cuja função é gerenciar dinamicamente os nomes simbólicos dos processos e seus respectivos endereços de rede (host:porta).

Descrição do Mecanismo Implementado

O Servidor de Nomes atua como um diretório central. Ele mantém um mapeamento entre nomes simbólicos (ex: “central”, “gerenciador_alimentadores”) e seus respectivos endereços de rede. Dessa forma, qualquer componente pode descobrir a localização de outro utilizando apenas seu nome, sem a necessidade de codificar manualmente endereços IP ou portas em seu código-fonte.

Registro e Consulta de Componentes

Cada componente que deseja ser localizado por outros precisa se registrar no Servidor de Nomes, enviando uma mensagem do tipo “registro”, contendo dois campos:

- ”nome”: identificador simbólico do processo

- (ex: “central”, “gerenciador_alimentadores”);

- ”endereço”: endereço IP e porta do processo no formato “host:porta”

- (ex: “localhost:11115”).

Esse registro é feito através do método registrar() da classe utilitária ComunicadorDeNomes.

Para encontrar outro componente, um processo utiliza o método consultar(), também da classe ComunicadorDeNomes. Essa operação envia ao servidor uma mensagem do tipo “consulta”, contendo apenas o campo “nome”, e recebe como resposta o endereço correspondente ou uma mensagem de erro (“nao-encontrado”) caso o nome não esteja registrado.

Informações Armazenadas e Forma de Acesso

As informações armazenadas pelo Servidor de Nomes consistem em um mapa de pares chave-valor (Map<String, String>), onde:

- A chave é o nome simbólico do processo (por exemplo: “central”);

- O valor é o endereço IP e porta do processo (por exemplo: “localhost:11115”).

Esse mapeamento é mantido em memória e acessado diretamente para responder às requisições de consulta. A comunicação entre os processos e o servidor de nomes ocorre via sockets TCP, utilizando mensagens no formato JSON para garantir padronização e extensibilidade.

Impacto no Sistema

A introdução desse mecanismo torna o sistema:

Mais flexível: componentes podem ser alterados de porta ou máquina sem necessidade de modificar os demais.

Mais organizado: a comunicação ocorre por nomes simbólicos, facilitando manutenção e leitura do código.

Mais realista: simula a descoberta de serviços presente em arquiteturas distribuídas reais, como DNS, JNDI ou service registries em microsserviços.

Este mecanismo foi essencial para permitir a expansão da arquitetura implementada nesta etapa, viabilizando a integração entre Central de Controle, Sensores e Gerenciador de Alimentadores de forma robusta e modular.

Instruções de Construção e Execução do Projeto

Este projeto, denominado FazendaInteligente, foi desenvolvido em Java utilizando a versão 24 da JDK e a biblioteca externa org.json (json-20230227.jar) para manipulação de mensagens em formato JSON. A estrutura está organizada por pacotes funcionais, facilitando a modularização dos componentes do sistema distribuído.

Pré-requisitos

Antes de executar o projeto, certifique-se de que os seguintes itens estejam devidamente instalados e configurados no seu ambiente de desenvolvimento:

- JDK 24 ou superior instalada
- Ambiente de desenvolvimento Java compatível (recomendado: NetBeans, IntelliJ ou Eclipse)
- Biblioteca externa json-20230227.jar (já incluída no repositório)
- Git (para clonar o repositório)

Obtenção do Projeto

O projeto está disponível publicamente no GitHub. Para obtê-lo, utilize o seguinte comando no terminal:

```
git clone https://github.com/otaviomcortez/Sistemas-Distribuidos.git
```

Estrutura de Pastas

O projeto está organizado da seguinte forma:

- `centraldecontrole/` – contém a `CentralDeControle.java`

- `gerenciadordealimentadores/` – contém o `GerenciadorDeAlimentadores.java`
- `nomeservidor/` – contém o `ServidorDeNomes.java`
- `sensoresambientais/` – contém os `SensoresAmbientais.java`
- `sensoresdogado/` – contém os `SensoresDoGado.java`
- `util/` – contém a classe `ComunicadorDeNomes.java` (utilitário comum de registro e consulta)
- `lib/` – contém o arquivo `json-20230227.jar`

Compilação do Projeto

Certifique-se de que o arquivo `json-20230227.jar` esteja adicionado ao class-path do projeto.

Compile todos os arquivos Java. Em ambiente de terminal (ex: `cmd` ou `bash`), estando na raiz do projeto, você pode usar:

```
javac -cp lib/json-20230227.jar src/**/*.java
```

Execução dos Componentes

Os componentes devem ser iniciados em janelas de terminal separadas, seguindo a seguinte ordem:

Servidor de Nomes:

```
java -cp lib/json-20230227.jar src/nomeservidor/ServidorDeNomes
```

Central de Controle:

```
java -cp lib/json-20230227.jar src/centraldecontrole/CentralDeControle
```

Gerenciador de Alimentadores:

```
java -cp lib/json-20230227.jar src/gerenciadordealimentadores/GerenciadorDeAlimentadores
```

Sensores Ambientais:

```
java -cp lib/json-20230227.jar src/sensoresambientais/SensoresAmbientais
```

Sensores de Gado:

```
java -cp lib/json-20230227.jar src/sensoresdogado/SensoresDoGado
```

Observações Importantes

- Certifique-se de que a porta 11000 (Servidor de Nomes) não esteja em uso no sistema.
- Caso execute o projeto via IDE, verifique se o classpath da biblioteca `json-20230227.jar` está corretamente configurado nas propriedades do projeto.
- O projeto foi projetado para rodar localmente (localhost). Em ambientes distribuídos reais, ajustes nos endereços IP podem ser necessários.